Multi-Vendor Procurement & Purchase Order Management System

1. Project Title and Team Members

Project Title: Multi-Vendor Procurement & Purchase Order Management System

Team Number: Team 3

Team Members: Nisarg Shah, Smit Patel, Parva Shah, Swet Patel, Priya

2. Project Overview and Objective

Description of the project: This project is a comprehensive, full-stack enterprise procurement platform designed to simulate a real-world purchasing workflow. It digitally manages the entire lifecycle of how an organization purchases goods from external vendors, integrating role-based access control, real-time budget tracking, and extensive audit logging.

Overall goal of the database system: The primary goal is to digitize and streamline the multi-step, multi-department procurement process. By replacing manual paperwork with a centralized relational database, the system ensures data integrity, enforces strict department budget constraints, maintains a consistent state machine for all purchase requests, and provides actionable KPI metrics for management.

3. Database Design

Business rules summary: The system enforces several critical business rules. Managers can only approve requests if the department has sufficient remaining budget. Users are strictly segregated by roles (Employee, Manager, Procurement, Vendor, Finance), and actions are restricted accordingly. Every status change in the procurement lifecycle is immutably logged in a history table.

ERD and schema explanation: The database follows a normalized relational design (Third Normal Form) consisting of 16 interconnected tables divided into six functional groups: Authentication, Procurement Workflow, Vendor Management, Order Fulfillment, Finance, and System Monitoring. Key tables include Users, Departments, Purchase_Requests, Purchase_Orders, and Invoices.

Key design decisions: Data redundancy is minimized by strictly defining foreign key relationships. Roles, Departments, and Vendor Categories are separated into distinct tables. A dedicated PR_Status_History table is used to create a robust audit trail of state changes, rather than merely overwriting a status column.

4. Data Dictionary Overview

Description of tables and relationships:

- **Users & Roles:** Manages user authentication and permissions. Users belong to Departments and are assigned specific Roles.
- **Departments:** Tracks budget allocations and current expenditure.
- **Purchase_Requests & Approvals:** Stores item requests generated by employees and the subsequent approval/rejection decisions made by managers.
- **Purchase_Orders & Goods_Receipt:** Tracks official orders sent to vendors and records when physical goods are delivered.
- **Invoices & Payments:** Manages the financial settlement of completed orders.

Key attributes and constraints: Primary keys utilize AUTOINCREMENT integers. Foreign keys enforce referential integrity across all entities (e.g., a Purchase Order cannot exist without a valid Vendor and Purchase Request). NOT NULL constraints ensure critical data fields are always populated. Dates and timestamps are systematically recorded in US/Central time for audit consistency.

5. SQL Implementation

Overview of table creation scripts: The database was implemented using SQLite3. DDL scripts define tables with appropriate data types (INTEGER, TEXT, REAL, BOOLEAN) and establish foreign key constraints to maintain strict relational integrity.

Description of query categories:

- **Basic:** Standard INSERT and SELECT queries to manage user sessions and display initial dashboard data.
- **Intermediate:** Multi-table JOINs used to compile comprehensive views, such as linking Purchase Orders to their respective Vendors and original Purchase Requests.
- **Advanced:** Complex transactional queries that cascade status updates backwards across the workflow (e.g., updating a Purchase Request to 'PAID' when a Finance user clears an Invoice) and aggregate functions (SUM, COUNT) used to dynamically calculate remaining department budgets and KPI metrics.

Explanation of selected queries: A critical implemented query is the budget deduction during the manager approval phase. The system performs a SELECT to verify the current 'budget_used' against 'budget_allocated'. If sufficient funds exist, an UPDATE adjusts the department's budget, followed by an INSERT into the PR_Approvals and PR_Status_History

tables, executed within a single logical transaction to prevent race conditions.

6. Frontend Integration (Optional Extra Credit)

Frontend tool/framework: The frontend is built using HTML5, Custom CSS3 (implementing a modern dark theme with glassmorphism), and Bootstrap 5.3 for responsive layouts. Data visualization is powered by Chart.js. The application uses the Python Flask framework with Jinja2 for server-side template rendering.

Database connection: The Flask backend connects to the SQLite database using Python's built-in sqlite3 library. A custom `get_db_connection()` function establishes the connection and utilizes the `sqlite3.Row` row factory to return records as dictionary-like objects, which are then passed seamlessly into Jinja2 templates for dynamic HTML rendering.

Screenshots showing interaction: Live data is retrieved from the database and displayed as interactive KPI cards and data tables on role-specific dashboards. (Note: Visual screenshots of the frontend interface are provided in the separate project presentation slides).

7. Challenges and Solutions

Issues encountered: One major challenge was maintaining data consistency across the multi-stage procurement state machine. Initially, updating a Purchase Order's status did not reflect on the Employee's original Purchase Request, leading to disjointed data views. Additionally, enforcing strict budget constraints required careful transactional handling.

How the team addressed them: We implemented a cascading status update mechanism in the backend routing. When a Vendor marks an order as 'SHIPPED', or Finance marks an invoice as 'PAID', the backend automatically traces the foreign keys backward to update the original Purchase Request and logs the transition in the history table. Budget constraints were resolved by validating the exact department funds dynamically before executing any approval queries.

8. Conclusion

Summary of the project: The Multi-Vendor Procurement system successfully digitizes a complex corporate workflow into an intuitive, secure, and highly responsive web application. It effectively demonstrates the power of relational databases in enforcing business logic and maintaining data integrity across distributed roles.

Key learnings: The team gained significant experience in database normalization, managing foreign key dependencies, and executing complex SQL queries within a web application environment. We also learned the importance of comprehensive audit logging and transactional consistency.

Possible future improvements: Future enhancements could include migrating the backend database from SQLite to PostgreSQL for better scalability, implementing automated email

notifications for pending approvals using SendGrid, and building an external API endpoint to allow vendor systems to interface directly with the database.